

Your Agent Didn't Fail. Your Harness Did.

Chinese teaching note prepared by Codex

2026-08-05



视频作者/频道: AI Engineer

发布日期: 2026-07-29

视频时长: 18:25

视频链接: <https://www.youtube.com/watch?v=BIInpv7lGp1o>

学习定位: 面向工程实践的中文教学笔记, 保留关键英文术语与原始判断。

目录

导读：先给结论	2
1 静默成功：交付正常，事实却已丢失	2
1.1 最危险的故障为何看起来像成功	2
1.2 模型、生产编排层与持久记录	3
1.3 对话记录与回执回答不同问题	4
1.4 本节小结	4
2 生产契约：模型提议，Harness 提交，回执证明	4
2.1 三方契约如何划清责任	4
2.2 汽车类比：能力需要方向、制动与黑匣子	5
2.3 从事件入口到运行回执的蓝图	5
2.4 设计审查应追问哪些边界	7
2.5 本节小结	7
3 状态重建与具名所有权	7
3.1 每一轮都在重新装配世界	7
3.2 Telegram 事件暴露的状态缺口	8
3.3 System of record：谁有权定义现实	9
3.4 把“记住”改写成可验证问题	9
3.5 本节小结	10
4 有序变更：保留并行能力，封住丢失更新	10
4.1 两个正确写入，为何会得到一个错误结果	10
4.2 核心不变量：一个可变边界，一条有序提交路径	11
4.3 从用户体验反推提交边界	12
4.4 本节小结	12
5 生命周期终结性：每一次等待都必须结束	12
5.1 沉默为何是生产故障	12
5.2 截止时间、看门狗与取消的职责分工	13
5.3 为什么“自动重试”仍可能无限等待	14
5.4 本节小结	14
6 有界权限：让每一次执行都匹配当前审批	14
6.1 可请求的动作不等于可执行的动作	14
6.2 审批漂移：旧点击为何会变成新风险	14
6.3 把审批建模为有界执行状态	15
6.4 让权限证明穿过重试、回放与结果记录	16
6.5 本节小结	16

7 证明必须抵达用户可见边界	16
7.1 内部成功为何仍可能让用户失望	16
7.2 证明是一条链	17
7.3 五类故障都可以沿边界定位	18
7.4 工程落点：先定义“到达”	18
7.5 本节小结	18
8 五问运行回执审计	18
8.1 从架构原则转向一条真实链路	18
8.2 第一问：什么唤醒了它	19
8.3 第二问：它继承了什么状态	19
8.4 第三问：它使用了哪项权限	20
8.5 第四问：实际执行了什么	20
8.6 第五问：什么证据留存下来	20
8.7 把五问应用于开场事故	20
8.8 本节小结	20
9 总结：拥有状态、有序变更、证明行动	21
9.1 从单轮能力走向跨轮次可靠性	21
9.2 五条控制线如何互相支撑	21
9.3 说话者结尾保留的判断	22
9.4 可执行的收尾方法	22
9.5 本节小结	22

导读：先给结论

这场演讲的核心判断是：生产级代理的可靠性取决于完整 Harness，而不取决于模型单轮是否给出了流畅回答。Harness 必须拥有状态，按顺序提交变更，为外部工作设置明确终态，约束当前权限，并用回执证明结果已经抵达用户真正关心的边界。

全篇主张

模型提出候选行为；生产编排层负责状态、顺序、生命周期与权限；持久回执连接触发、执行和用户可见结果。缺失其中任一边界，局部成功都可能被误判为交付成功。

下面九节沿着同一条学习路径展开：先看“看起来成功却丢失事实”的事故，再拆解状态所有权、并发提交、终结性、审批绑定与用户可见证明，最后收束为五问审计方法。

1 静默成功：交付正常，事实却已丢失

本节的结论是：一条回复出现在用户界面，只能证明某次交付抵达了可见边缘；只有负责该事实的系统能够在后续轮次、重启或重放中重新构造它，系统才拥有持久事实（durable truth）。因此，生产可靠性需要同时检查用户体验与持久记录，不能只看模型回答是否流畅。

1.1 最危险的故障为何看起来像成功

演讲以一个退款记忆事件建立问题边界。用户要求智能体记住某位客户的退款事实，助手回复“已经记录”；界面没有报错，当前轮次也正常结束。下一轮需要使用该事实时，系统却无法重建它。用户经历了成功，后续系统继承的现实已经不完整。

显式崩溃会留下停止边界：操作者看到错误，能够回到最后一个已知良好状态。静默成功缺少这条边界。用户没有理由怀疑回复，监控也可能没有明显告警，下一轮的模型仍能围绕残缺历史生成连贯语言。连贯性由当前输入驱动，它不能反向证明历史完整。

流畅回答无法证明记忆完整

“模型仍然答得很好”只说明模型能够利用眼前工作集生成连贯输出。若退款事实没有进入可重放的持久记录，后续流畅回答只是建立在残缺历史之上。静默成功会同时误导用户、操作者与自动评估。

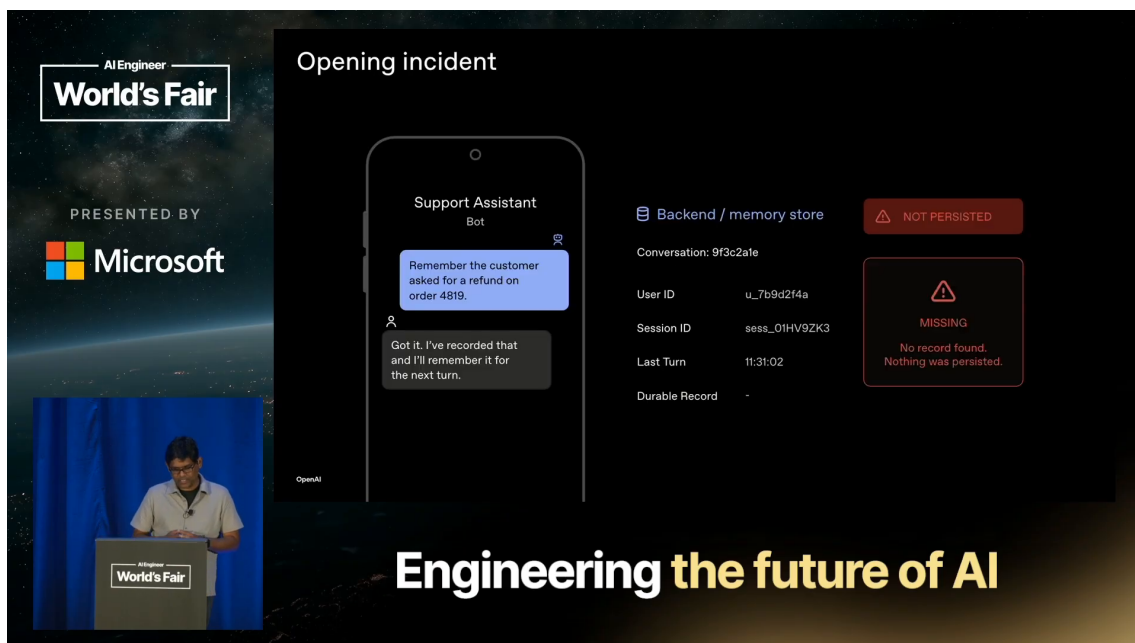


图 1：开场故障画面：支持助手声称已记录退款事实，后端 / 记忆存储区却显示 NOT PERSISTED、MISSING，且没有持久记录。¹

1.2 模型、生产编排层与持久记录

模型（model）是根据生产编排层提供的工作集提出文本、工具调用或动作的概率组件。它可以生成一条合理的确认语，也可以建议写入某个事实。模型输出仍处在提议阶段，尚未证明权限检查、外部执行或持久化已经完成。

Harness（生产编排与控制层）负责围绕模型组织生产行为，包括事件路由、状态重建、权限判断、提交变更、生命周期终止与证据记录。后文统一简称为“生产编排层”或 harness。这里的划分用于定位责任：模型负责提出候选行为，生产编排层负责把被允许的行为推进到受控边界。

退款事件由此可以拆成三层事实：

1. **模型层**：模型生成了“已记录”的文本，或者提出了记录退款事实的动作。
2. **交付层**：消息抵达用户界面，用户确实看见了确认回复。
3. **持久层**：负责记忆或业务状态的系统保存该事实，并能在下一轮重放。

前两层成功仍允许第三层失败。生产系统若把“文本已经发出”当作“事实已经保存”，就会把不同边界压成一个模糊的成功标志。

可靠记忆的判定

一个事实只有在具名系统能够持久保存并通过既定重放路径恢复时，才算被系统可靠记住。当前回复、临时上下文和看似健康的日志都不能单独替代这项判定。

¹视频画面时间区间：00:00:32-00:01:16。

1.3 对话记录与回执回答不同问题

对话记录说明智能体说了什么。回执 (receipt) 是跨越当前轮次仍然存在的证据, 说明系统允许了什么、尝试了什么、执行了什么, 以及相关边界最终确认了什么。二者都重要, 证据能力各有边界。

以退款记忆为例, 对话记录可以证明助手曾承诺“已经记录”; 它无法证明记忆存储接受了写入, 也无法证明下一轮的状态重建会读取这条记录。可靠回执需要把动作与实际持久边界关联起来, 例如保存对象的身份、结果状态、所属系统与可重放标识。若保存失败, 回执应记录明确失败, 使用户看到的结果与持久状态保持一致。

两个互补视角

用户界面回答“人看见了什么”, 持久记录回答“系统以后能恢复什么”。生产诊断需要同时保留这两个视角: 前者保护体验, 后者保护连续性。任何一侧缺失, 系统都可能把局部成功误报为整体成功。

由此得到本讲的第一条工程原则: 一次生产动作应当拆成可验证的边界结果, 成功状态应以真正拥有该事实的系统为准。模型质量依然重要; 它覆盖的是提议质量, 无法独立承担状态持久性与可重放性。

1.4 本节小结

静默成功的危险来自证据错位: 用户看见了回复, 持久记录却没有形成。模型可以在残缺历史上继续保持语言连贯, 因此输出质量无法暴露这一缺口。可靠系统需要区分可见交付、对话记录、持久事实与回执, 并要求具名系统通过可重放路径证明事实仍然存在。下一节将把这项诊断展开为模型、生产编排层与回执之间的生产契约。

2 生产契约: 模型提议, Harness 提交, 回执证明

本节的结论是: 可用的智能体系统需要把能力、控制与证据分配给不同角色。模型产生提议 (proposal); 生产编排层在权限、状态和顺序约束下提交变更 (commit); 回执保存能够跨轮次检查的结果证据。这份契约把“回答看起来合理”转换为“生产动作可以追踪、重放与审计”。

2.1 三方契约如何划清责任

模型可以建议消息、工具调用、文件编辑或命令。提议是候选行为, 它尚未经过当前权限判断, 也尚未在外部系统产生效果。生产编排层持有生产边界: 它检查状态转换是否合法、当前主体是否有权执行、同一可变边界上的提交顺序是否成立。提交变更完成后, 回执保留这次运行真正发生了什么。

生产三方契约

模型提出候选行为; 生产编排层决定并提交受控变更; 回执证明允许、尝试、执行与边界确认的结果。

这三个角色形成一条责任链。任何环节都不能用前一环节的成功代替后一环节的证据。

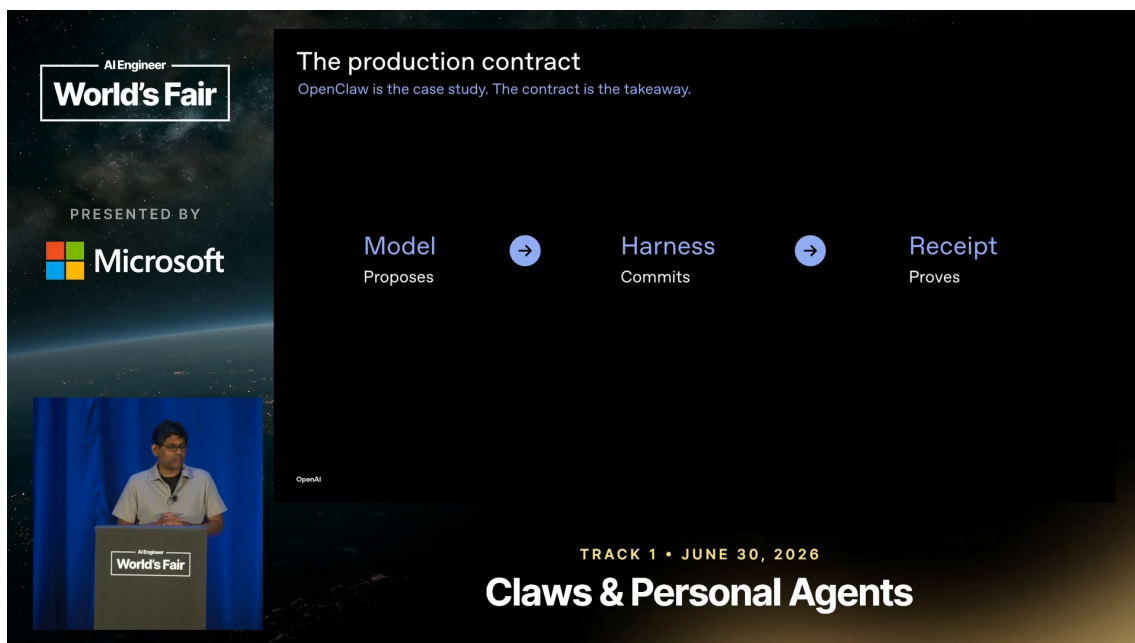


图 2: 画面中的生产契约: Model 提议 (Proposes), Harness 提交 (Commits), Receipt 证明 (Proves)。²

提交变更的关键含义是“让被接受的状态变化在所属边界成为持久结果”。消息发送、日历更新、工单状态变化与代码写入分别拥有不同的提交边界。生产编排层需要把每个动作送到正确边界，并保存边界返回的结果。单一的“工具调用成功”字段若没有说明由谁确认、确认了哪一层，就无法承担生产契约。

2.2 汽车类比：能力需要方向、制动与黑匣子

讲者用汽车解释 harness 的角色：模型像发动机，决定系统可产生多强的能力；生产车辆还需要转向、刹车、道路规则、仪表盘和黑匣子。转向对应事件与任务路由，刹车对应取消、截止时间和安全策略，道路规则对应权限与提交约束，仪表盘提供当前运行状态，黑匣子保存可审计证据。

类比的边界

汽车类比帮助区分“能力”与“生产控制”，它不代表智能体具有车辆式的固定动力学。模型输出具有概率性，工具与外部系统也有各自失败模式；工程设计仍需逐一说明状态、权限、生命周期和证据边界。

强发动机提高加速能力，缺少制动会扩大风险。相同逻辑适用于智能体：更强模型能够提出更复杂的计划，也能触达更多动作面；harness 若没有同步加强控制与观测，系统只是更快地抵达一个无法解释的状态。

2.3 从事件入口到运行回执的蓝图

生产蓝图从事件（event）开始。事件可以来自聊天消息、webhook、定时器、心跳、工具结果或另一个外部系统。控制面先把事件映射到会话键；会话键确定状态边界，并把相关工作送入对应

² 视频画面时间区间：00:02:39–00:03:07。

会话通道。运行时随后装配工作集，调用模型与工具，执行策略和审批检查，再把允许的变更送入有序提交路径。

这条路径可以按职责拆成七步：

1. **接收事件**：保存触发类型与唯一身份，避免后续只能猜测“谁唤醒了系统”。
2. **路由会话**：由控制面计算会话键，使同一状态边界的事件进入一致路径。
3. **装配工作集**：读取对话记录、会话状态、记忆、策略版本与工具定义。
4. **生成提议**：模型基于当前工作集提出文本或动作。
5. **检查执行条件**：生产编排层验证策略、审批、参数和权限范围。
6. **提交边界变更**：把允许的动作送到所属系统，并在需要时保证有序提交。
7. **写入审计轨迹**：把触发、继承状态、权限、执行尝试与边界结果连接成运行回执。

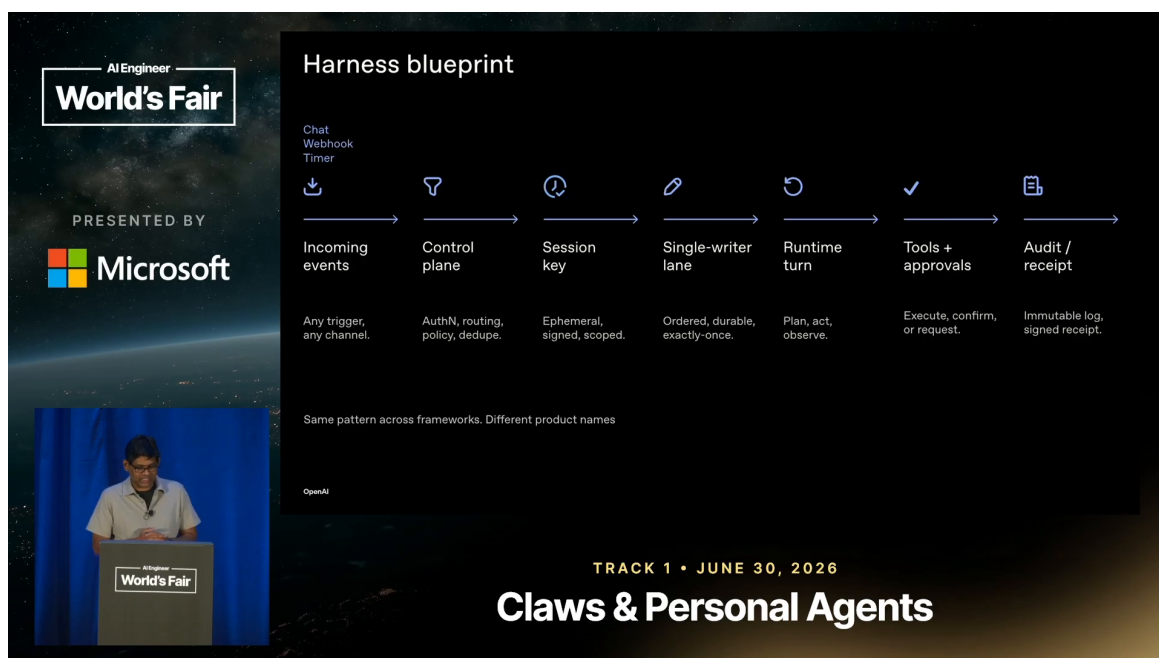


图 3: 画面中的 Harness 蓝图：传入事件依次经过控制面、会话键、单写入者通道、运行时轮次、工具与审批，最终进入审计 / 回执。³

审计轨迹（audit trail）指能够跨轮次存活并连接整条运行路径的证据。它与调试日志互补：日志可以包含丰富的局部细节，审计轨迹需要稳定回答触发、状态、权限、执行与结果之间的关系。若某个工具产生大量日志，却没有可关联的动作身份和边界结果，操作者仍然无法判断运行是否真正完成。

局部成功不等于契约完成

模型生成成功、工具进程返回零、消息 API 接收请求和界面最终显示内容，属于不同边界。生产契约要求逐层保存结果，并让回执明确终止于当前动作真正关心的边界。

³视频画面时间区间：00:03:36–00:05:03。

2.4 设计审查应追问哪些边界

这份蓝图的价值不在于固定组件名称，而在于迫使团队标出责任归属。每加入一种事件源，都要说明它如何映射到会话键；每加入一种可变状态，都要说明提交路径；每加入一个工具，都要说明权限检查与终止结果；每向用户承诺一次成功，都要说明哪份回执能够证明该承诺。

蓝图还揭示一项隐藏依赖：模型看到的世界由生产编排层组装。路由错误、状态遗漏、策略版本过期或工具定义不一致，都可能让模型基于错误工作集产生合理提议。改进提示词无法修复这些边界，因为缺失发生在提示词形成之前或动作提交之后。

2.5 本节小结

生产三方契约把智能体运行拆成提议、提交与证明。模型提供能力，生产编排层管理事件、状态、权限与变更，回执保存跨边界证据。Harness 蓝图进一步把事件、会话键、状态边界、运行时、策略、工具和审计轨迹连接起来。下一节将深入工作集如何重建，以及每个持久事实为何必须拥有具名所有者。

3 状态重建与具名所有权

本节的结论是：智能体运行时通常不会像人类一样持续记忆，它在每一轮依赖生产编排层重新组装工作集。每个未来可能使用的重要事实都需要具名的状态所有权（ownership）和可验证的重放路径；存储位置只回答字节在哪里，所有权回答哪个系统有权定义并重建现实。

3.1 每一轮都在重新装配世界

状态重建（state rehydration）是为当前轮次重新组装对话记录、会话状态、记忆、策略与工具定义的过程。模型只会看到生产编排层提供的内容。某项输入缺失或过期时，模型仍可能生成语法正确、逻辑顺滑的回答，因此回答连贯性不能证明工作集完整。

可以把状态重建理解为程序启动时从多个持久源加载配置和数据。一次 Python 函数调用只会读取传入参数与可访问状态；若调用方漏传一个关键参数，函数仍可能正常返回，只是结果建立在错误输入上。智能体的差异在于，工作集同时包含自然语言历史、外部记忆、策略和动态工具面，缺口更难从表面输出识别。

状态重建的核心约束

生产编排层应能说明当前轮次使用了哪些对话记录、会话状态、记忆、策略版本和工具定义。任何影响决策的事实若无法追溯到持久来源，都会形成潜在的状态缺口。

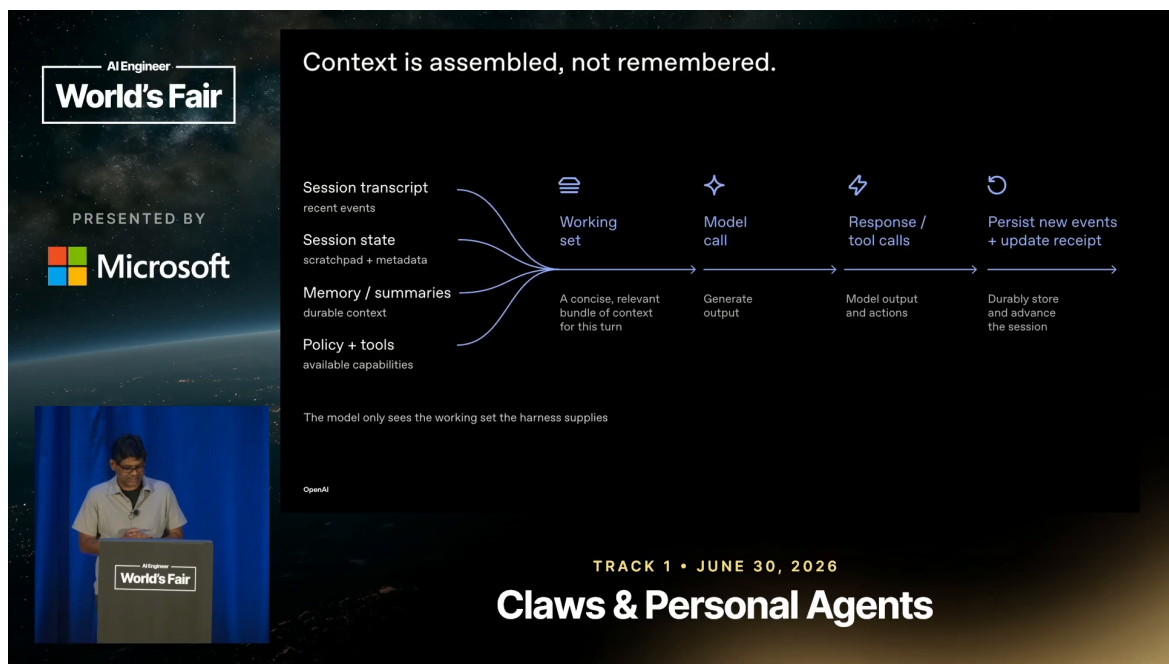


图 4: 画面中的状态重建流程：会话记录、会话状态、记忆 / 摘要以及策略与工具汇入工作集；模型调用产生回复 / 工具调用后，新事件被持久化并更新回执。⁴

状态重建还依赖事件路由。相同用户可能拥有多个会话，相同会话可能由聊天、回调或定时器触发。控制面若把事件映射到错误会话键，即使所有底层数据都存在，当前轮次仍会读取错误世界。由此可见，“已保存”和“会被正确重建”是两个连续条件。

3.2 Telegram 事件暴露的状态缺口

讲者用一个公开案例说明“已交付”与“已记住”的分离：Telegram 回复可以成功发送，相关 router turn 却没有写入活跃上下文或会话记录。用户看见回复，日志看起来健康，下一轮仍缺少这次变化的持久记录。成功发送只能证明发送路径完成，无法证明未来上下文包含该轮次。

该案例支持的是边界诊断：缺失发生在路由轮次进入持久上下文的路径。材料没有提供 Open-Claw 的完整内部实现，因此本文不推断具体数据结构、事务机制或普遍故障概率。可复用的教学结论是，外部交付与会话持久化需要分别确认，并由回执关联。

“日志健康” 仍可能遗漏未来上下文

局部日志通常围绕当前组件记录。发送组件可以如实报告消息已交付，同时会话所有者没有收到对应轮次。诊断时应沿回执检查事实是否进入其 system of record，以及重放路径是否真的读取它。

这类缺口会在后续轮次放大：模型根据残缺对话继续推理，用户以为系统记得先前承诺，自动评估可能只检查当前回复。直到某个需要历史事实的动作出现，错误才变得可见。延迟暴露使根因离用户投诉更远，也提高了重放与审计难度。

⁴ 视频画面时间区间：00:05:08–00:05:43。

3.3 System of record: 谁有权定义现实

状态所有权指持久状态能够定义某项事实、并能重建该事实的系统身份。这里的“所有者”指系统，不指某位自然人。演讲给出一组具体映射：

- 日历事件属于日历系统；
- 客服状态属于工单系统；
- 代码变更属于工作区或代码仓库；
- 对话轮次属于会话记录；
- 用户偏好属于记忆存储。

这些系统构成各自事实的 system of record，即权威记录系统。生产编排层可以缓存、索引或复制事实来加速工作集装配，但复制品不能悄然取得定义现实的权力。若日历缓存与日历系统冲突，应由明确的一致性规则决定如何处理；缺少规则时，系统会在不同轮次继承不同现实。

存储位置与状态所有权

存储位置说明字节物理或逻辑上放在哪里；状态所有权说明哪个系统的持久状态具有解释权，以及如何从该系统重建事实。一个事实可以有多个副本，仍应只有一个明确的权威来源和重放路径。

所有权也不同于访问权限。某个 harness 可能获准读取日历事件，读取权不会使它成为日历事实的所有者。它若需要修改日历，应通过日历系统的提交边界完成变更，并保存该边界返回的回执。这个区分让数据来源、执行权限与持久真相保持可解释。

3.4 把“记住”改写成可验证问题

“系统记住了吗”过于模糊。设计审查可以把它改写为两个具体问题：未来可能使用的每个事实由谁拥有？系统准备怎样重放它？这两个问题把记忆从体验词汇转换为可检查的工程契约。

一条合格的事实路径至少需要以下信息：

1. **事实身份**：要保存的对象与版本是什么；
2. **具名所有者**：哪个权威记录系统定义它；
3. **提交结果**：写入是否被所属系统接受；
4. **重放路径**：下一轮通过哪个键、查询或会话装配步骤恢复它；
5. **失败可见性**：保存或重放失败时，用户与操作者会看到什么终止结果。

所有权判定问题

对于智能体未来可能使用的每个事实，应当能回答：“谁拥有它，系统将如何重放它？”若没有任何具名所有者能够重放该事实，系统就没有可靠地记住它。

该方法还揭示未知依赖。缓存失效策略、会话键稳定性、策略版本固定方式、外部系统可用性和跨系统一致性都会影响重建结果。工程团队需要明确这些依赖的失败行为，避免把暂时可访问的副本误认成长期权威。

3.5 本节小结

状态重建决定模型在当前轮次看见什么，状态所有权决定未来轮次能够恢复什么。Telegram 案例表明，消息发送成功与会话事实持久化可以分离。可靠设计需要为日历、工单、代码、对话与偏好等事实指定 system of record，并记录可执行的重放路径。存储、权限和所有权回答不同问题；三者只有在提交与回执中被明确连接，系统才拥有可持续的记忆基础。

4 有序变更：保留并行能力，封住丢失更新

4.1 两个正确写入，为何会得到一个错误结果

当系统已经知道“谁拥有这份状态”之后，下一个生产问题是：谁可以修改它，修改以什么顺序生效。单次调用看起来完全正确，并不能保证多个调用叠加后仍然正确。演讲用一句高度概括的话指出这个风险：两个局部正确的写入，仍可能产生一个全局错误的结果。

最典型的情形是“读取—修改—保存”竞争。设持久状态最初为 S_0 ：

1. 写入者 A 读取 S_0 ，得到本地副本，并把记录 a 更新为 a' ；
2. 几乎同时，写入者 B 也读取同一个 S_0 ，在另一份本地副本中把记录 b 更新为 b' ；
3. A 先保存，持久状态暂时包含 a' ；
4. B 随后用自己的完整副本保存。由于这份副本仍带着旧的 a ，A 的更新被安静地覆盖。

这里没有畸形请求，也没有明显报错。A 与 B 各自在自己的输入快照上完成了合理操作；错误产生在共享状态的提交边界。用户最终可能看到已经撤销的事项重新出现，或收到重复的后续动作。系统表面像“忘了”，根因却是提交顺序没有受到约束。

“最后写入者获胜”会隐藏数据丢失

“最后写入者获胜”只描述了覆盖发生后的表象，没有回答被覆盖的变更是否允许丢失。把它当作默认一致性策略，等于让调度时机替系统作业务决定。

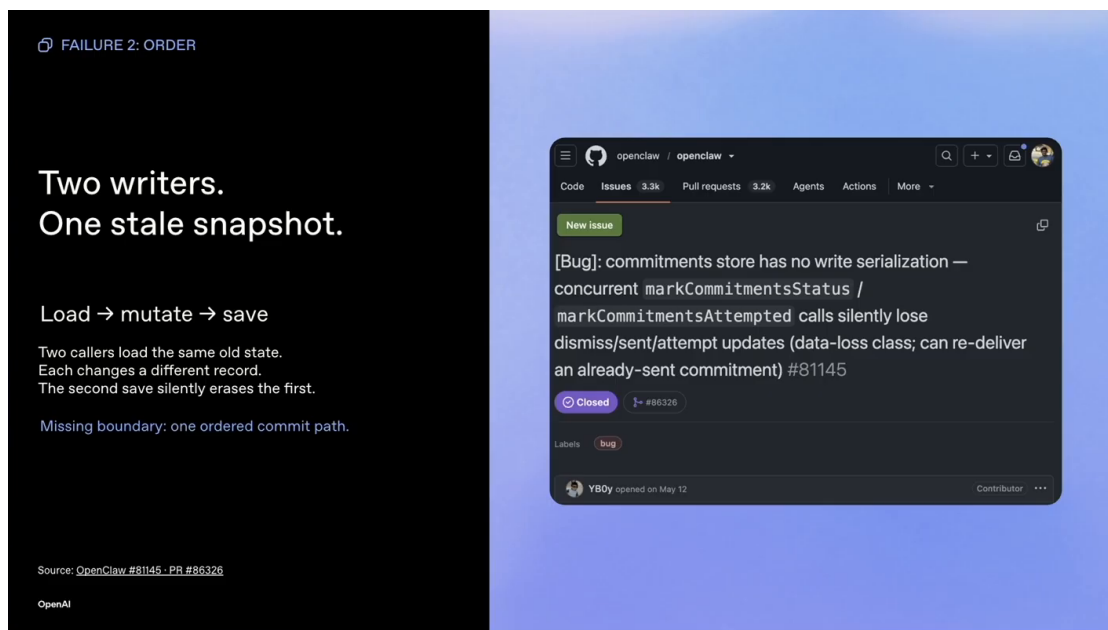


图 5：画面显示两个写入者从同一旧快照出发：各自修改不同记录，第二次保存却静默抹去第一次；幻灯片将缺失边界标为“一条有序提交路径”。⁵

4.2 核心不变量：一个可变边界，一条有序提交路径

解决方向并非把整套系统变成串行程序。演讲明确保留了并行子代理、并行读取、相互独立的检索以及同时运行的多个会话。真正需要收窄的是改变同一份可变状态的时刻。

有序提交不变量

对每一个可变状态边界，所有提交变更都必须经过一条明确的有序提交（ordering）路径。并行可以发生在准备阶段；最终改变共享事实的步骤必须有可解释的先后次序。

这条规则把并发控制放在最有信息增益的位置：读取与推理通常可以扩展，提交变更则由状态所有者协调。实现机制可以是队列、互斥机制、事务或作用域明确的锁（lock）。选型依赖状态边界及底层存储的能力，演讲的约束停留在架构层：保护提交时刻，避免把锁扩大到整个系统。

单写入通道（single-writer lane）是理解这一边界的直观模型。它表示某个可变状态在提交阶段只有一个排序入口；它不表示整套应用只能运行一个任务。不同状态边界仍可拥有各自的通道，读取任务也可继续并行。

排序、锁与幂等性分别解决什么

有序提交防止相互重叠的写入覆盖彼此；锁或事务是实现这种协调的候选机制；幂等性（idempotency）约束重试，使同一个已识别尝试被重复执行时不会额外制造一次副作用。三者相互配合，但含义并不等同：幂等重试无法自动合并两个不同写入者对同一旧快照所做的更新。

⁵ 视频画面时间区间：00:08:28–00:08:57。

4.3 从用户体验反推提交边界

用户看不到队列、锁或事务，他们直接体验行为。刚刚纠正过的事实又恢复旧值，会被理解为“健忘”；一条被永久占用的通道，会被理解为“卡死”；系统内部先宣布完成、外部效果随后才出现，则会显得前后矛盾。因此，有序提交既是数据一致性约束，也是产品行为的一部分。

工程检查可以沿着以下问题展开：

- 哪个持久化边界拥有这份可变事实？
- 哪些调用可能从同一旧版本开始修改？
- 提交时由谁排序，冲突如何显式暴露？
- 重试是否携带稳定的幂等标识，并能关联到既有回执？
- 锁定范围是否只覆盖必要的提交临界区？

这些问题把“偶尔忘记”还原为可检查的状态竞争。只要系统无法指出唯一的提交路径，就无法证明两个局部正确的操作不会相互抹除。

4.4 本节小结

丢失更新发生在共享状态的提交边界：多个写入者读取同一旧状态，各自正确修改，后一次保存却覆盖前一次结果。可靠 harness 应允许读取、检索与独立任务并行，同时让每个可变状态边界只有一条有序提交路径。队列、事务、互斥机制和锁是候选手段；幂等性负责约束重试。用户最终体验的是排序后的行为，因此提交顺序必须成为显式产品规则。

5 生命周期终结性：每一次等待都必须结束

5.1 沉默为何是生产故障

生产系统中的等待不能无限悬置。一次外部调用发出后，如果对应结果永远不会到达，而运行仍被记作“进行中”，后续消息就可能全部排在这段沉默之后。用户看到的现象十分简单：代理不再响应。系统内部却可能没有任何失败记录，因为它仍在等待一个已经不可能出现的事件。

演讲以悬空工具调用为例：会话里存在工具调用记录，却没有匹配的工具结果。进程可能已经退出，连接可能已经中断，超时也可能发生在结果写入之前。精确原因对诊断很重要；从生命周期角度看，公共故障形态是同一个——运行依赖一个永远不会到达的事件。

终结性不变量

沉默不是终态。每一个外部边界上的尝试都必须收敛为显式结果：成功、失败、超时、取消或达到最大尝试次数；对应回执必须记录这一终态。

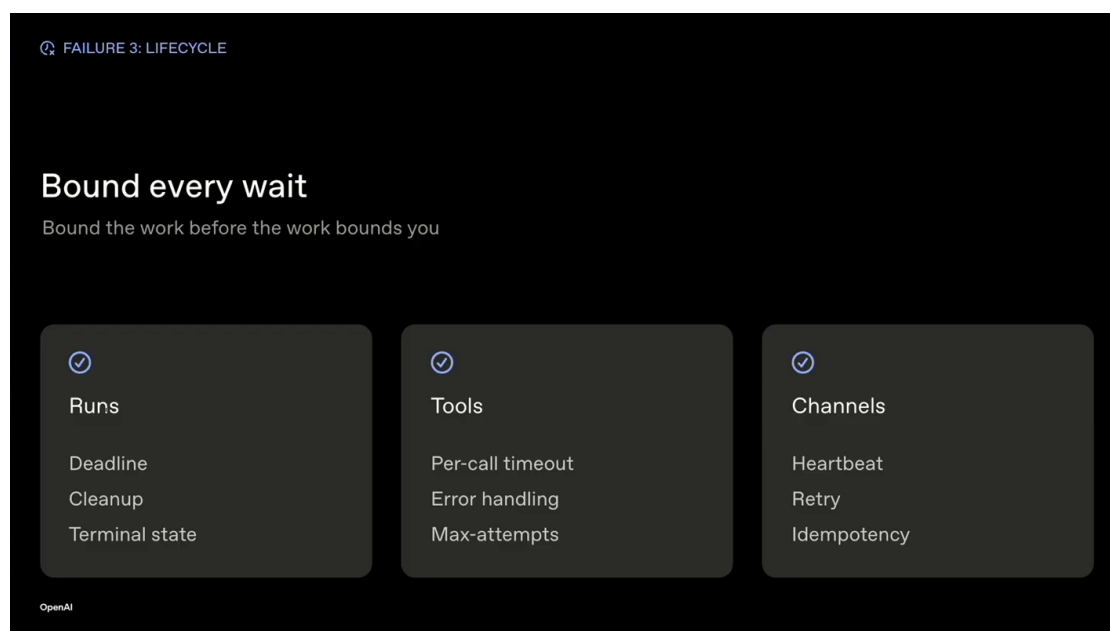


图 6: 画面将有界等待分为三层：运行层包含截止时间、清理与终态，工具层包含单次调用超时、错误处理与最大尝试次数，通道层包含心跳、重试与幂等性。相邻讲解补充：回执记录最终终态，使后续步骤无需猜测。⁶

5.2 截止时间、看门狗与取消的职责分工

截止时间（deadline）为一次外部等待规定最晚结束点。它表达的是生命周期边界，不能被当作一般的性能指标：调用可以很慢，只要仍在明确边界内；越过截止时间后，系统必须产生新的控制决策。

看门狗负责观察。它让停滞工作变得可见，在超过预期窗口或失去进展信号时触发检查。工具层还需要把超时和错误表示为正常可消费的结果，使上层不必从“没有消息”猜测发生了什么。取消则是显式终结控制：它结束一个有界尝试，并把取消结果写入记录。

四个相邻概念的边界

截止时间规定“最晚等到何时”；超时是越过等待上限后产生的结果；看门狗负责发现或上报停滞；取消负责主动终止有界尝试。它们都服务于终结性，却不能互相替代。

一个完整生命周期控制面通常需要以下环节：

1. 在调用开始时记录稳定的尝试身份、起始时间与截止时间；
2. 让工具成功、工具错误、超时和取消都返回结构化结果；
3. 用看门狗暴露失去进展的工作，并限制最大尝试次数；
4. 为恢复操作保留独立控制路径，使恢复命令不会继续排在被卡住的工作后面；
5. 把最终状态写入回执，让下一步基于持久事实继续执行。

⁶视频画面时间区间：00:10:45–00:11:16。

以上列表是依据演讲机制整理的教学重构，具体状态名与调度实现需要由实际系统定义。关键约束是每条路径最终都落到持久、可重建的结果。

5.3 为什么“自动重试”仍可能无限等待

重试只有在尝试身份、停止条件与结果记录都明确时才安全。缺少最大尝试次数时，失败会变成循环；缺少幂等性时，迟到结果与新尝试可能制造重复副作用；缺少取消传播时，上层宣布终止，底层工作仍可能继续运行。生命周期终结性因此必须和上一节的有序提交、幂等性以及回执关联起来。

恢复命令不能被同一条堵塞通道困住

如果“取消”“跳过”或“恢复”也必须等待悬空工作释放同一通道，系统虽然提供了恢复按钮，实际上仍没有恢复路径。控制操作需要能够抵达并终结目标尝试。

回执在这里承担关键交接作用。它记录终态后，后续步骤无需根据日志碎片猜测前一次调用是否仍在运行，也无需把沉默解释成成功。演讲强调“先约束工作，否则工作会反过来约束系统”，其工程含义正是：把时间边界变成可执行、可观察、可审计的生命周期规则。

5.4 本节小结

悬空工具调用的危险在于，它让运行永久停留在非终态，并把后续消息堵在一个不会完成的等待之后。截止时间限定等待，看门狗暴露停滞，工具超时与错误形成结构化结果，取消与最大尝试次数负责终结，独立恢复路径保证控制命令能够抵达。最终回执记录成功、失败、超时、取消或尝试耗尽，使下一步不必猜测。

6 有界权限：让每一次执行都匹配当前审批

6.1 可请求的动作不等于可执行的动作

当聊天开始触发真实动作，可靠性问题就进入权限边界。模型可以根据上下文提出调用工具的请求，也可以解释为什么需要执行；这些能力仍然只是提议。真正的执行权必须由 harness 依据当前身份、策略与审批状态作出决定。

权限边界

模型可以请求动作，生产编排层负责决定动作是否在当前上下文中获得执行授权。可请求性不构成权限；历史上曾经允许过，也不构成当前权限。

这一区分阻止语言模型把“能描述一个动作”“能生成调用参数”或“记得用户点过同意”误当作可执行凭证。模型可以推理边界，但不能成为边界本身。

6.2 审批漂移：旧点击为何会变成新风险

审批漂移（approval drift）指审批被重用时，其动作、范围、身份或有效期已经不再匹配原始授权。演讲描述了一类审批回调问题：过期的已审批状态在重启后仍被保留并再次处理，随后阻塞了通道中的工作。历史点击确实存在，当前有效权限却已经不存在。

该案例揭示了一个常见错误：把审批当作“人类曾经靠近系统并点击过确认”的模糊记忆。审批应当是一份有作用域的执行状态，始终绑定到它授权的具体动作；到期应进入终态，不能循环等待或自动扩大授权。由于自动字幕在具体实现措辞上存在歧义，本节只保留演讲明确支持的审批生命周期机制，不推断相关产品的内部实现。

历史同意不能替代当前授权

重试、回放、重启或通道回调如果丢失了原审批的身份、参数、范围与期限，系统就无法证明即将执行的动作仍是当初获批的动作。此时应停止执行并重新取得有效授权。

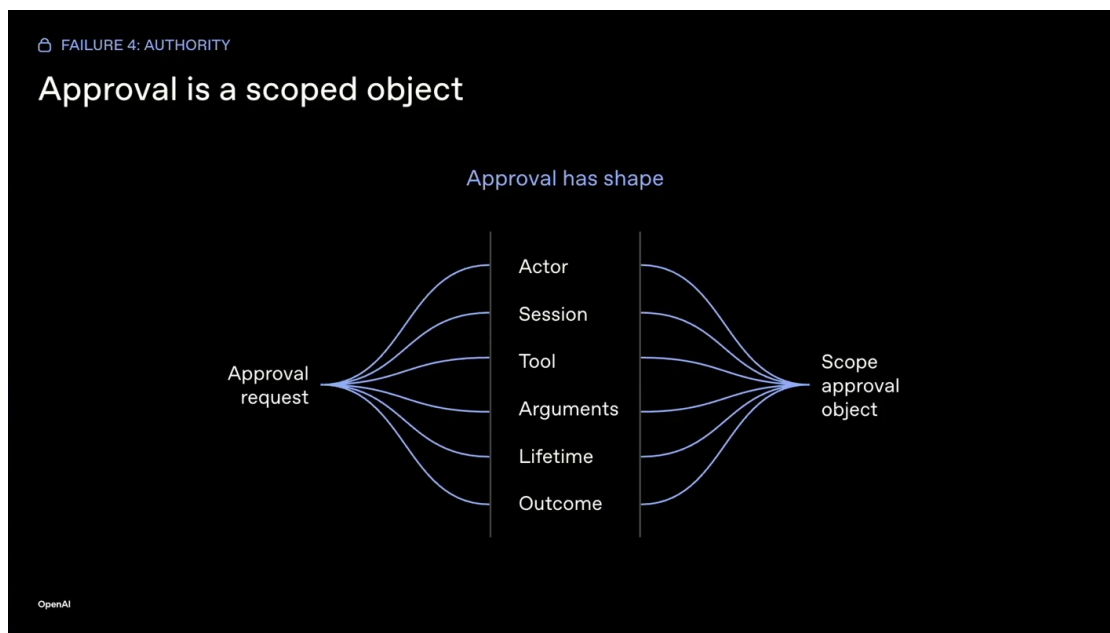


图 7: 画面将批准请求塑造成有作用域的批准对象, 并列出 Actor、Session、Tool、Arguments、Lifetime 与 Outcome 六项字段。相邻讲解补充: 该对象还应指向回执, 重试、重放或回调不得丢失这些字段。

7

6.3 把审批建模为有界执行状态

有界权限 (bounded authority) 要求执行权受到身份、会话或运行、工具、参数、作用域和生命周期的共同约束。演讲给出的审批对象应能回答以下问题:

- 谁批准了该动作, 批准发生在哪个会话与哪次运行?
- 哪个执行者将使用这份权限?
- 被批准的是哪个工具、哪些具体参数与哪个资源范围?
- 权限从何时开始, 到何时失效?
- 执行最终得到什么结果, 对应哪一份持久回执?

⁷ 视频画面时间区间: 00:12:07–00:12:30。

这些字段共同建立“当初批准的动作”与“现在准备执行的动作”之间的同一性。重试或回放需要携带同一审批对象的有效绑定；参数、身份或范围发生实质变化时，应视为新动作并重新判定。有效期届满则产生明确的过期终态，使生命周期控制和权限控制在同一处闭合。

有界权限的三层防线

最小权限缩小可调用的工具与资源表面；作用域凭证确保具体动作使用正确身份并受资源范围约束；审批与审计分别记录执行前为何允许、执行后实际发生了什么。三层共同作用，单独依赖一次人类点击无法覆盖完整执行链。

6.4 让权限证明穿过重试、回放与结果记录

权限判断不能只在模型生成工具调用时出现一次。真实系统还会经历排队、重试、进程重启、事件回放和外部回调。每次准备执行时，harness 都需要确认审批仍然属于当前运行、当前执行者和当前参数；执行结束后，再把实际结果与审批身份写进回执。

这种设计也限定了审计能证明什么。审计轨迹应连接触发事件、继承状态、有效权限、执行尝试、外部结果与回执。若链条中只剩“用户以前点过同意”，系统只能证明发生过一次点击，无法证明当前动作拥有授权。

执行前后都要有证据

执行前，系统证明当前动作落在有效审批范围内；执行后，回执证明该权限被用于哪个尝试、产生了什么结果。审批约束将要发生的事，审计轨迹保留已经发生的事。

6.5 本节小结

模型生成动作提议的能力不等于执行权限。审批漂移发生在旧审批被带入新的身份、参数、作用域或生命周期后继续使用。可靠 harness 把审批建模为有界执行状态，绑定审批者、会话或运行、执行者、工具、参数、范围、期限、结果与回执；过期必须终结，变化必须重新判定。最小权限、作用域凭证、审批与审计共同保证系统能够证明“当前执行的动作就是当前获批的动作”。

7 证明必须抵达用户可见边界

7.1 内部成功为何仍可能让用户失望

工具返回 `success`，只能说明某个内部组件接受或完成了调用。用户真正关心的结果还可能停在更远处：消息是否出现在聊天界面，工单是否真的更新，文件是否形成预期内容，日历事件是否能够再次查询。只观察组件结果，会把链路中途的局部事实误当成最终事实。

演讲中的“缺失边缘证明”案例呈现了这种错位：消息工具报告发送成功，普通助手回复也能够出现，目标消息却没有渲染在用户界面。系统随后可能声称“已经发送”，用户也完全可能如实回答“从未看到”。两者记录了不同边界上的事实。

Failure boundary

内部成功只证明内部路径走到某一步。若回执没有连接用户实际关心的外部效果，`success` 仍然可能掩盖一次交付失败。

7.2 证明是一条链

可靠证明需要沿生产路径逐段建立：模型先产生提议，策略层决定允许或拒绝，执行器发起实际调用，用户可见边界确认结果或明确记录失败，最后由持久回执保存整条链。该顺序可以压缩为：

User-visible proof

模型提议（proposal）→ 策略决定 → 执行尝试 → 用户可见边界确认或明确失败 → 持久回执（receipt）。

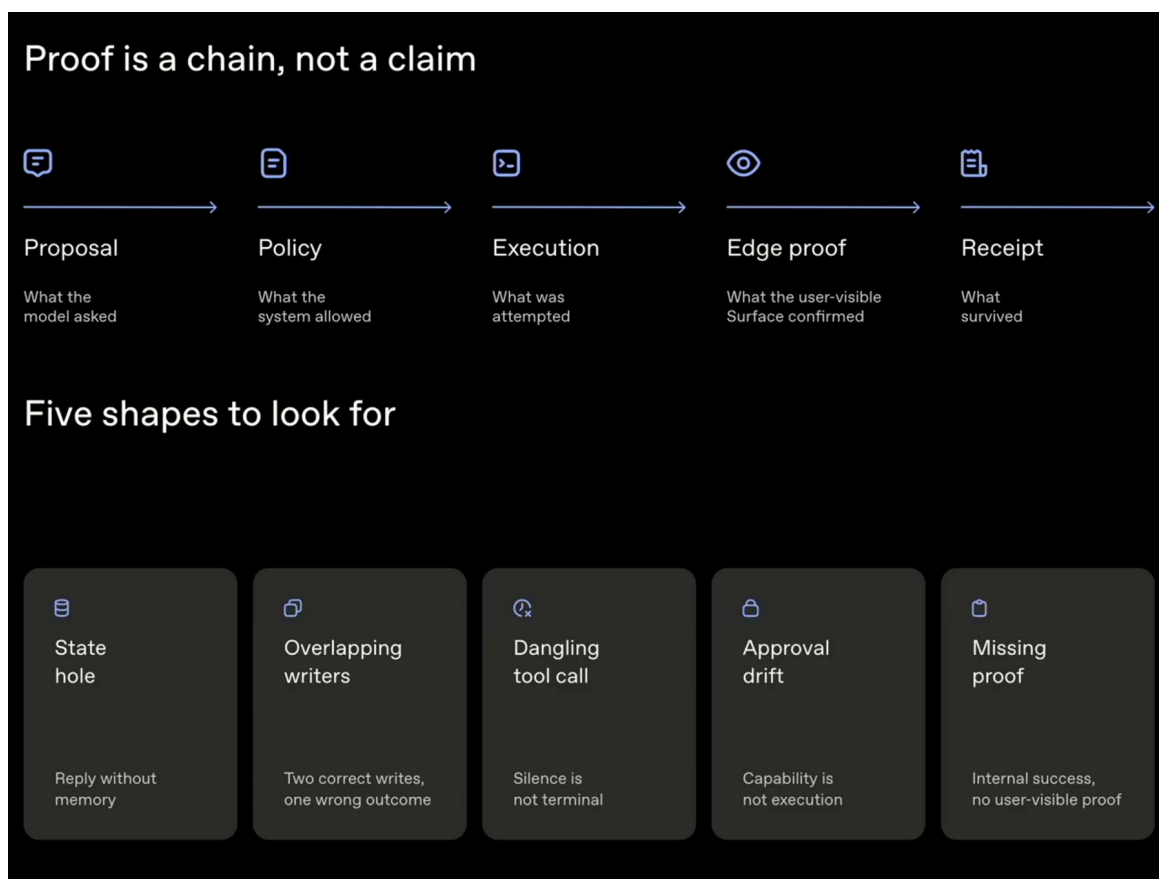


图 8：源视频证据链的教学重构：Proposal、Policy、Execution 与 Edge proof 连接到 Receipt；下半部分归纳状态空洞、重叠写入、悬空工具调用、审批漂移与证明缺失五类失败形状。⁸

这条链区分了三类经常混淆的证据。对话记录保存系统说过什么；工具结果保存某个组件报告了什么；回执保存系统允许了什么、尝试了什么、执行了什么，以及在相关外部边界确认了什么。证据越接近用户关心的效果，越能够支持生产结论。

用户可见证明（user-visible proof）因此具有明确边界：它证明效果到达用户实际依赖的外部表面。对消息系统而言，边界可能是消息已渲染并可查询；对工单系统而言，边界可能是目标字段在权威系统中已经更新；对文件操作而言，边界可能是目标文件存在且内容可重新读取。具体边界由产品承诺决定，不能由内部组件自行缩短。

⁸ 视频画面时间区间：00:13:41--00:14:29；图为源视频两张连续幻灯片的教学拼接。

7.3 五类故障都可以沿边界定位

演讲把前文问题归纳为五种故障形态：状态缺口、重叠写入、悬空工具调用、审批漂移和缺失边缘证明。它们表面症状不同，诊断时都可以追问三个问题：用户看到了什么，哪一条边界被破坏，完整回执原本能够捕获什么。

Proof chain

状态缺口破坏跨轮次重建；重叠写入破坏变更顺序；悬空调用破坏生命周期终止；审批漂移破坏权限有效性；缺失边缘证明破坏外部效果确认。五者都属于 Harness（生产编排与控制层）应当显式管理的边界。

这种定位方式能够避免把所有异常归因于模型推理。若模型提出了合理动作，而状态没有持久化、写入被覆盖、等待永不结束、权限已经过期，或结果没有抵达用户界面，提高模型能力不会自动修复这些边界。

7.4 工程落点：先定义“到达”

设计回执前，需要先写清楚每类动作的完成条件。完成条件应落在可验证的外部事实上，并同时允许明确失败。随后，系统才能把策略决定、尝试编号、执行结果和边界确认连接为一条可重放的审计轨迹。

若外部系统只能提供“已接受”而无法提供“已呈现”，回执也应忠实记录证据上限，例如“服务端已接收，界面呈现未确认”。这种表述保留不确定性，避免将弱证据升级为强结论。

7.5 本节小结

证明必须终止于用户实际关心的边界。对话记录与组件结果都是有用证据，其证明范围有限；完整回执把提议、策略、执行和外部确认连接起来。生产系统应先定义每种动作的外部完成条件，再让成功、失败与未确认状态都能被持久记录。

8 五问运行回执审计

8.1 从架构原则转向一条真实链路

系统图可以说明组件如何连接，真实运行回执才能暴露边界在哪里断裂。演讲给出的实践方法很克制：先选择一个代理系统中的一条真实生产路径，沿该路径取得回执，再回答五个固定问题。较小的审计范围能够保留触发、状态、权限、执行和证据之间的因果联系。

Five audit questions

五问审计依次检查：什么唤醒了运行；它继承了什么状态；它使用了哪项权限；实际执行了什么；最终留下了什么证据。



图 9: 运行回执的五问审计：逐项记录触发、继承状态、执行权限、实际调用与边界证据；无法回答的位置就是 Harness 的生产缺口。图为教学重构。⁹

这五问把“对话看起来流畅”转换为“生产运行可以检查”。每个问题对应一段责任边界，答案缺失的位置通常就是最值得继续诊断的位置。

8.2 第一问：什么唤醒了它

触发源可能是用户消息、Webhook、定时器、工具结果、子代理事件或重放。审计记录至少应包含触发类型与稳定身份，使系统能够判断两个事件是同一次交付的重试，还是两次独立请求。

触发身份缺失会直接削弱去重与排序判断。系统看到两个相似事件时，既无法证明应该合并，也无法证明应该分别执行；后续的幂等键与尝试编号也失去可靠起点。

8.3 第二问：它继承了什么状态

需要记录的继承状态包括对话记录、会话状态、记忆快照、策略版本和可用工具表面。模型只能基于 Harness 为当前轮次组装的工作集进行推理，因此一段连贯回答并不能证明输入完整。

这一问还应连接状态所有权：每个重要事实由哪个权威系统持久化，恢复时从哪里重建，读取的是哪个版本。若事实只出现在临时对话或局部日志中，重启与重放之后可能形成状态缺口。

State evidence

“日志里曾经出现”不能替代“权威状态可以重建”。审计需要记录继承状态的来源与版本，才能区分模型判断问题和工作集缺失问题。

⁹视频画面时间区间：00:14:42--00:16:18；图为源视频多张审计幻灯片的教学重构。

8.4 第三问：它使用了哪项权限

权限记录应覆盖执行者、会话或运行、工具、参数、作用域和有效期。模型请求某个动作，只表达了候选意图；真正执行还需要与该动作绑定的有界权限。

审批对象应指向一个待执行动作，并在重启、重试或上下文变化后重新验证其身份与寿命。历史点击若已经过期、参数已经变化或执行主体已经改变，就不能继续证明当前动作得到授权。

8.5 第四问：实际执行了什么

审计应记录实际工具或 API 调用、参数、尝试编号、幂等键以及外部系统返回的结果。这些字段描述真实副作用边界，能够与模型原先表达的意图进行核对。

幂等键用于识别同一逻辑尝试的重复执行；尝试编号用于区分一次动作中的多次外部调用。两者配合，既能防止重试产生额外副作用，也能保留每次尝试的结果。记录仍需继续延伸到外部边界，因为“调用已接受”未必等于“用户目标已完成”。

8.6 第五问：什么证据留存下来

最后一问检查用户真正关心的结果：工单是否更新，消息是否渲染，文件是否发生可验证变更，日历事件是否存在。回执应在这一外部事实处结束，并保留明确成功、明确失败或尚未确认的状态。

Audit trail

五问形成一条审计轨迹 (audit trail)：触发身份解释运行为何开始，继承状态解释模型看到了什么，有界权限解释系统为何可以执行，调用记录解释实际发生了什么，边界证据解释用户最终得到了什么。

8.7 把五问应用于开场事故

开场案例中，用户消息唤醒了运行；系统生成并交付了一条回复；后续轮次却无法从持久状态重建相关事实。沿五问检查，可以看到触发存在、通道发送发生、交付证据也可能存在，断点落在持久状态所有权与跨轮次重建上。交付留存了，状态没有留存，这个差距正是 Harness 故障。

该案例也说明审计顺序的价值。若诊断只停在“消息已经发出”，系统会把运行判断为成功；当第五问与第二问同时出现时，审计能够区分即时可见交付和持久事实，指出未来轮次为何继承了不完整现实。

8.8 本节小结

五问审计以一条真实生产路径为单位，依次检查触发、继承状态、有界权限、实际执行和存活证据。它把流畅对话还原为可检查的运行链，并通过缺失字段定位破损边界。实践时应先完整审计一条真实轨迹，再决定是否把同一方法扩展到整个系统。

9 总结：拥有状态、有序变更、证明行动

9.1 从单轮能力走向跨轮次可靠性

演讲的结论可以压缩为三项动作：拥有状态，有序变更，证明行动。模型能够改善单轮中的提议质量；生产可靠性还取决于 Harness 是否能够在多个轮次、重试、外部调用和重启之间保持一致事实。

Production contract

Own the state, order the mutation, prove the action: 为每个事实指定可重放的所有者，为每个可变边界建立有序提交路径，并用抵达外部边界的回执证明行动结果。

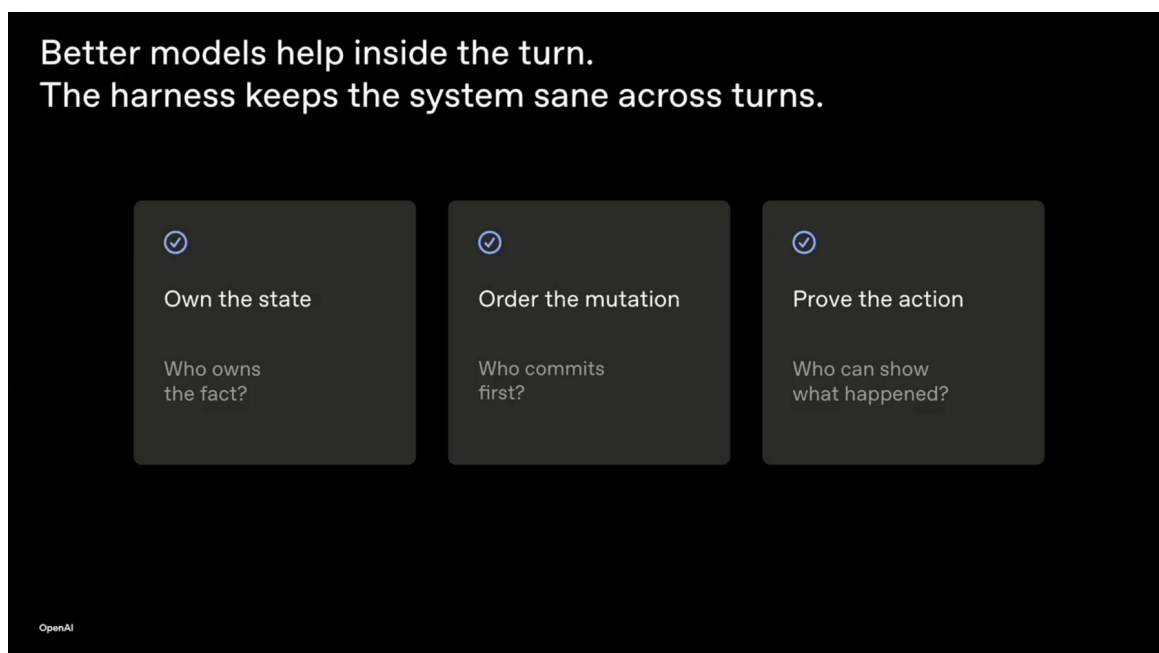


图 10: 源视频收束画面展示三条生产控制线: Own the state、Order the mutation、Prove the action; 模型能力改善单轮推理, Harness 负责跨轮次可靠性。¹⁰

三项动作覆盖了生产链的核心风险。状态所有权防止事实在交付后消失；有序提交防止两个局部正确的写入互相覆盖；外部证明防止内部成功被误报为用户成功。它们共同把模型提议转化为可以追责、恢复和验证的系统行为。

9.2 五条控制线如何互相支撑

完整的生产检查还需要把前文五条控制线连接起来：

1. **状态所有权**：每个重要事实都有权威系统与重放路径，避免状态缺口。
2. **变更顺序**：每个可变状态边界只有一条有序提交路径，避免丢失更新。
3. **有界工作**：每个外部等待都有截止时间、取消或其他明确终态，避免悬空运行。

¹⁰ 视频画面时间区间：00:16:55--00:17:41。

4. **受约束权限**：每次执行都绑定主体、动作、参数、范围和寿命，避免审批漂移。
5. **存活证据**：回执连接触发、状态、权限、执行与用户可见结果，避免缺失边缘证明。

这些控制线具有前后依赖。没有状态所有权，排序对象会变得含糊；没有明确终态，回执无法闭合；没有有界权限，成功执行也可能属于越权动作；没有用户可见证据，内部链路仍无法证明产品承诺已经兑现。

9.3 说话者结尾保留的判断

说话者在结尾再次强调生产契约：“A model proposes, the harness commits, and the receipt proves it.” 当文本能够触发真实行动时，评估问题随之扩展：系统是否能够拥有状态、安排变更顺序、限制工作时长、约束权限，并保存证据。

这段结尾把单轮回答与生产服务区分开来。代理循环可以回答一轮；Harness 才承担跨轮次服务所需的编排与控制责任。模型推理仍然重要，其输出只是行动链的起点，不能单独证明系统已经安全、持久地完成任务。

演讲随后给出两个深入方向：OpenAI Agents SDK 与说话者关于生产代理系统的 Agent Stack 资料。此处保留其学习指向，不推断具体 SDK 已经实现了演讲中的全部机制；实际采用时仍应核对当前版本、运行环境与产品边界。

Scope limitation

框架或 SDK 的存在不能自动证明某个生产系统具备完整状态所有权、提交顺序、终态、权限与回执。每项保证都需要在真实运行路径中验证。

9.4 可执行的收尾方法

最小可行行动是选取一条真实轨迹，取得其运行回执，并逐项回答五问。若某一问只能依赖临时日志、口头假设或组件自报成功，该位置就应被视为证据缺口。修复后，再用同一轨迹执行重试、重启或外部失败场景，检查事实是否仍能重建、工作是否能够终止、权限是否仍然匹配、结果是否抵达用户边界。

Repair sequence

实践顺序可以保持简单：先审计一条真实路径，定位一个破损边界，补齐该边界的所有权或证据，再用可重复场景验证。局部闭环形成后，系统才适合扩展到更多工具、渠道与并发路径。

这一方法也明确了评估重点。模型基准回答“模型能否推理”；生产轨迹回答“整个系统能否可靠行动”。两类评估共同存在，后者必须覆盖跨轮次状态、并发变更、生命周期、执行权限和用户可见结果。

9.5 本节小结

生产级代理的核心责任落在完整 Harness：模型提出候选行动，控制层在有界权限下组织执行与有序提交，持久回执证明外部结果。最终检查可以归结为三句话：拥有状态，有序变更，证明行动；并用五问审计在一条真实轨迹上验证这些承诺。